

RUN-WAY Fashion Platform — Comprehensive Technical Report

1. Executive Summary

RUN-WAY (formerly codenamed *Aphronique*) is a full-stack social fashion platform that enables designers to upload and showcase their haute couture collections, while visitors can discover, rate, like, and follow designers in real time. The platform features role-based access control (Visitor, Designer, Admin), a real-time discovery feed, content moderation tools, and a responsive React-based user interface.

Project Repository: <https://github.com/Regueroo29/RUN-WAY.git>

2. System Architecture & Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + Vite	SPA (Single Page Application), fast HMR builds
Backend	Node.js + Express.js	RESTful API server, Socket.IO gateway
Database	TiDB Cloud (MySQL-compatible)	Distributed relational database
Image Storage	Cloudinary	Cloud-based CDN for design images & avatars
Auth	JWT (jsonwebtoken) + bcrypt	Stateless token authentication, password hashing
Real-Time	Socket.IO	Bidirectional event-driven updates
Deployment	Render (Backend) + Vercel	Free-tier cloud hosting

Layer	Technology	Purpose
	(Frontend)	
Version Control	Git + GitHub	Source control & CI/CD

3. Database Architecture

3.1 Database Platform: TiDB Cloud

The project uses **TiDB Cloud**, a distributed, MySQL-compatible database-as-a-service. It provides horizontal scalability, high availability, and standard MySQL protocol compatibility, allowing the use of `mysql2` Node.js driver without modification.

3.2 Database Name

`runway_db`

(Originally named during early development; retained for consistency.)

3.3 Tables Overview (10 Tables)

#	Table	Purpose
1	<code>users</code>	Stores registered accounts (visitors, designers, admins)
2	<code>designs</code>	Fashion design posts (title, description, image, season)
3	<code>likes</code>	Many-to-many relationship: users liking designs
4	<code>ratings</code>	1-5 star ratings per user per design
5	<code>follows</code>	Many-to-many: visitors following designers
6	<code>activity_logs</code>	Audit trail of user actions (login, upload, like, etc.)
7	<code>moderation_logs</code>	Admin actions (hide, suspend, delete) with reasons

#	Table	Purpose
8	user_notifications	System alerts sent to users
9	comments <i>(if present)</i>	User comments on designs
10	<i>(additional utility tables)</i>	Session or config data

3.4 Primary Keys (PK)

Every table uses an **AUTO_INCREMENT integer** as its Primary Key. This ensures: - Each record has a unique identifier. - IDs are never reused, even after deletion (e.g., IDs 1, 2, 7, 26 indicate designs 3–6 were deleted). - MySQL automatically assigns the next sequential number.

Example:

```
user_id INT AUTO_INCREMENT PRIMARY KEY
design_id INT AUTO_INCREMENT PRIMARY KEY
log_id INT AUTO_INCREMENT PRIMARY KEY
```

3.5 Foreign Keys (FK)

Foreign Keys enforce **referential integrity** — preventing “orphan” records.

Child Table	Foreign Key	References	Meaning
designs.designer_id	→	users.user_id	Every design belongs to a user
likes.user_id	→	users.user_id	Every like is from a user
likes.design_id	→	designs.design_id	Every like is for a design
ratings.user_id	→	users.user_id	Every rating is from a user
ratings.design_id	→	designs.design_id	Every rating is for a design
follows.follower_id	→	users.user_id	Follower is a

Child Table	Foreign Key	References	Meaning
d			user
follows.designer_id	→	users.user_id	Designer being followed is a user
activity_logs.user_id	→	users.user_id	Every log entry belongs to a user

3.6 Composite Unique Constraints

Some tables prevent duplicate relationships:

likes table:

UNIQUE KEY unique_like (user_id, design_id)

→ A user cannot like the same design twice.

follows table:

UNIQUE KEY unique_follow (follower_id, designer_id)

→ A user cannot follow the same designer twice.

4. Backend Architecture (Node.js + Express)

4.1 Entry Point: backend/index.js

The main server file configures: - **Express** app with CORS, JSON parsing, and static file serving. - **MySQL connection pool** (TiDB Cloud with SSL in production). - **Multer** file upload middleware (local disk in dev, Cloudinary in production). - **Socket.IO** server attached to the HTTP server for real-time events. - **Admin routes** mounted at /api/admin.

4.2 Authentication Flow

1. Registration:

- Frontend sends first_name, last_name, email, password, role, gender, date_of_birth.
- Backend checks for duplicate email.

- Password is hashed with **bcrypt** (salt rounds: 10).
- Username is auto-generated from first + last name.
- Role is validated (visitor, designer, admin) but frontend only allows visitor or designer.
- Record inserted into `users` table.

2. Login:

- User sends email + password.
- Backend checks if account is suspended (with optional end date).
- Password is compared with `bcrypt.compare()`.
- **JWT token** is generated and returned.
- Activity logged: `INSERT INTO activity_logs (user_id, action_type) VALUES (?, 'login');`

3. Token Usage:

- Frontend stores token in `localStorage`.
- Every API request includes `Authorization: Bearer <token>` via Axios interceptor.

4.3 Key API Endpoints

Method	Endpoint	Description
POST	<code>/api/register</code>	Create new user account
POST	<code>/api/login</code>	Authenticate and receive JWT
GET	<code>/api/users/:id</code>	Get public profile + follower count
PUT	<code>/api/users/:id/profile</code>	Update bio, brand, social links
POST	<code>/api/users/:id/avatar</code>	Upload profile avatar
GET	<code>/api/designs</code>	Discovery feed (active designs + ratings + likes)
GET	<code>/api/designs/designer</code>	Get designs by specific designer

Method	Endpoint	Description
	<code>/:id</code>	
POST	<code>/api/designs</code>	Upload new design (multipart/form-data)
POST	<code>/api/designs/:id/update-with-image</code>	Update design with optional new image
DELETE	<code>/api/designs/:id</code>	Delete design (owner or admin only)
POST	<code>/api/likes/toggle</code>	Like/unlike a design
POST	<code>/api/ratings</code>	Rate a design (1-5 stars)
POST	<code>/api/follows/toggle</code>	Follow/unfollow a designer
GET	<code>/api/follows/check</code>	Check if following a designer
GET	<code>/api/health</code>	Server health check

4.4 Admin Routes (backend/routes/admin.js)

Protected by JWT-based `isAdmin` middleware.

Method	Endpoint	Description
GET	<code>/api/admin/users</code>	List all users with stats
GET	<code>/api/admin/designs</code>	List all designs with stats
GET	<code>/api/admin/activities</code>	Filtered activity log
GET	<code>/api/admin/users/:id/details</code>	User details + designs + moderation history
POST	<code>/api/admin/users/:id/suspend</code>	Suspend user with reason & duration
POST	<code>/api/admin/users/:id/unsuspend</code>	Reinstate suspended user
GET	<code>/api/admin/designs/:id/details</code>	Design details + moderation history
POST	<code>/api/admin/designs/:id</code>	Hide, unhide, or delete design

Method	Endpoint	Description
	d/moderate	
GET	/api/admin/notifications/:userId	Get unread notifications
POST	/api/admin/notifications/:id/read	Mark notification as read

4.5 Real-Time Events (Socket.IO)

The backend emits events on state changes:

Event	Trigger	Payload
design_uploaded	New design POST	Complete design object
design_deleted	DELETE design	{ design_id, designer_id }
design_updated	Admin hide/unhide	{ design_id, status }
design_liked	Like toggle	{ design_id, like_count }
design_rated	Rating added	{ design_id, avg_rating, rating_count }
account_suspended	Admin suspend	{ reason, until, message }

5. Frontend Architecture (React + Vite)

5.1 Build Tool: Vite

- Fast development server with Hot Module Replacement (HMR).
- Optimized production builds deployed to Vercel.

5.2 Routing (React Router)

Route	Page	Access
/	Home (Landing)	Public
/login	Login	Public

Route	Page	Access
/register	Registration	Public
/preview	Preview Experience	Public
/path	Role Selection	Public
/dashboard	Discovery Feed	Visitor / Designer / Admin
/designer	Designer Studio	Designer only
/designer/:id	Public Designer Profile	Public
/profile	User Profile	Authenticated
/admin	Admin Dashboard	Admin only

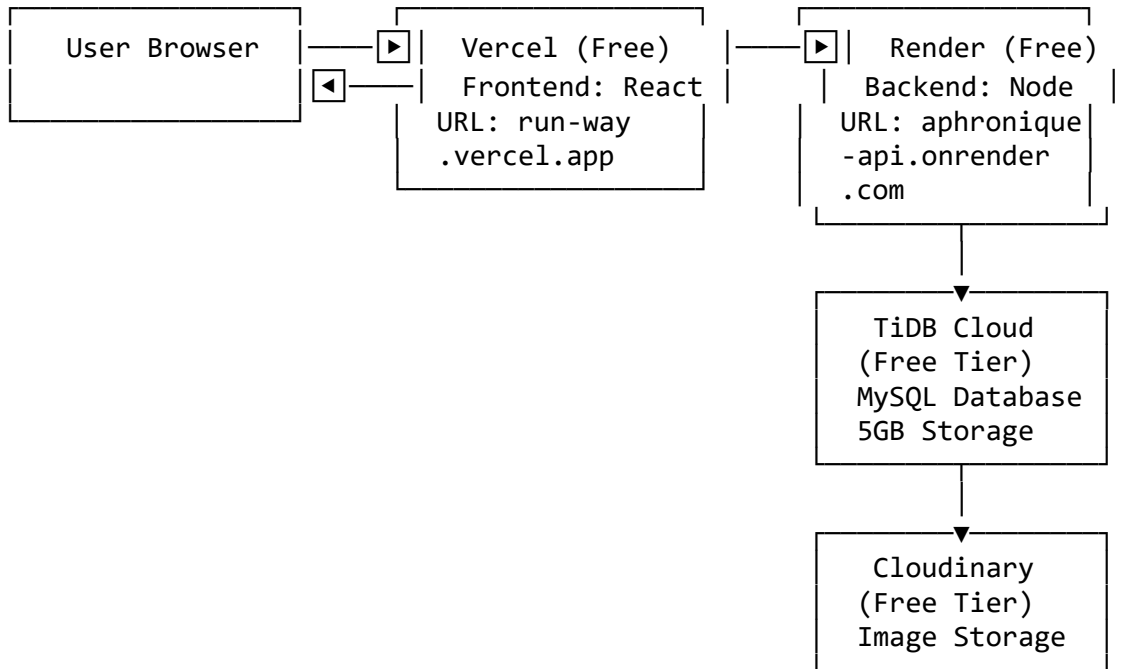
5.3 Key Components

- **Dashboard.jsx** — Discovery feed with trending/for-you/activity tabs, design modal, like/rate/follow actions.
- **Designer.jsx** — Designer studio: gallery, upload, edit, stats, profile management.
- **Admin.jsx** — Admin panel: user table, design table, activity log, moderation modals.
- **DesignDetailModal.jsx** — Admin modal for viewing/moderating a specific design.
- **UserDetailModal.jsx** — Admin modal for viewing/suspending a specific user.
- **SocketContext.jsx** — Global Socket.IO provider wrapping the app.
- **api.js** — Centralized Axios instance with automatic JWT injection.

5.4 State Management

- **Local React state** (useState, useEffect) for component-level data.
 - **localStorage** for persistent user session (token + user object).
 - **Socket.IO listeners** for real-time cross-tab updates.
-

6. Deployment Architecture



6.1 Render (Backend)

- **Service:** runway-api (renamed from aphronique-api; URL remains legacy).
- **Build:** Auto-deploy from GitHub main branch.
- **Environment Variables:** BASE_URL, DB_HOST, DB_USER, DB_PASSWORD, JWT_SECRET, CLOUDINARY_*, NODE_ENV=production.

6.2 Vercel (Frontend)

- **Project:** run-way.
- **Framework Preset:** Vite.
- **Environment Variable:** VITE_API_URL=https://aphronique-api.onrender.com/api.

6.3 TiDB Cloud

- **Type:** Serverless Tier (MySQL 8.0 compatible).
- **Connection:** SSL-enabled in production (rejectUnauthorized: false).
- **Storage:** 5GB free tier.

6.4 Cloudinary

- **Folder:** runway (production uploads).
- **Formats:** jpg, jpeg, png, webp, gif (max 5MB).
- **Transformation:** Auto-quality optimization.

7. Security Features

Feature	Implementation
Password Hashing	bcrypt (salt + hash)
Authentication	JWT signed tokens
CORS	Restricted to allowed origins (localhost, Vercel)
SQL Injection Prevention	Parameterized queries (? placeholders)
File Upload Validation	Multer: images only, max 5MB
Role-Based Access	Visitor / Designer / Admin permissions
Admin Route Protection	JWT verification + role check
Suspension System	Time-based or permanent account suspension

8. How Admin Accounts Are Created

Admin accounts CANNOT be created through the website registration form.

This is an intentional security design.

Regular Users

- Created via `/api/register` frontend form.
- Role limited to visitor or designer by frontend UI.
- Password hashed by bcrypt automatically.

Admin Accounts

- Created by running **SQL INSERT statements directly in MySQL Workbench** (or TiDB Cloud console).
- Alternatively, imported from the original database dump (`backup.sql`).
- Role is manually set to `'admin'`.

Example SQL:

```
INSERT INTO users (username, email, password, role, first_name, last_name)
VALUES (
  'admin',
  'admin@runway.com',
  '$2b$10$...hashed_password...',
  'admin',
  'System',
  'Admin'
);
```

Why? The frontend registration form only offers two buttons: **DESIGNER** and **VISITOR**. There is no “ADMIN” option, preventing random users from escalating privileges.

9. Development Tools & Workflow

Tool	Purpose
VS Code	Primary IDE (with GitLens, ESLint, Prettier)
MySQL Workbench	Database design, schema visualization, direct SQL execution
TiDB Cloud Console	Cloud database management, connection strings
Postman / Browser DevTools	API testing
Git + GitHub	Version control, collaboration, deployment hooks
Render Dashboard	Backend deployment logs, environment variables
Vercel Dashboard	Frontend deployments, analytics
Cloudinary Dashboard	Media library, upload presets

9.1 Local Development vs Production

Aspect	Local Dev	Production (Render)
Image	backend/uploads/	Cloudinary CDN

Aspect	Local Dev	Production (Render)
Storage	folder	
Image URL	http://localhost:5000/uploads/...	https://res.cloudinary.com/...
Database	Local MySQL / TiDB	TiDB Cloud
CORS	localhost:5173 allowed	run-way.vercel.app allowed

10. Project Highlights

1. **Real-Time Discovery Feed** — New uploads, likes, ratings, and deletions reflect instantly across all connected clients without page refresh.
2. **Role-Based Fashion Studio** — Designers have a dedicated studio page with gallery, upload, edit, stats, and profile management.
3. **Content Moderation** — Admins can hide, unhide, or permanently delete designs; suspend/unsuspend users with reasons and durations.
4. **Smart Feed Algorithm** — “Trending” tab scores designs by likes + ratings; “For You” shows recent posts from followed designers.
5. **Activity Logging** — Every significant action (login, upload, like, follow, rate, moderation) is recorded in `activity_logs` for audit trails.
6. **Responsive Image Handling** — Automatic Cloudinary upload in production with format validation and quality optimization.
7. **Account Suspension** — Time-based suspensions automatically expire; permanent suspensions require manual admin intervention.

11. Folder Structure

```

RUN-WAY/
├── backend/
│   ├── middleware/
│   │   └── auth.js           # JWT token generation/verification
│   ├── routes/
│   │   └── admin.js         # Admin-only API routes
│   ├── uploads/             # Local image storage (dev only)
│   └── .gitignore

```

```
|— index.js                # Main Express server
|— package.json
|— package-lock.json
|— frontend/
|   |— public/              # Static assets (images, icons)
|   |— src/
|       |— assets/          # Vite-processed assets
|       |— components/
|           |— DesignDetailModal.jsx
|           |— UserDetailModal.jsx
|           |— ImageCropper.jsx
|           |— ToastProvider.jsx
|       |— context/
|           |— SocketContext.jsx
|       |— pages/
|           |— Admin.jsx / Admin.css
|           |— Dashboard.jsx / Dashboard.css
|           |— Designer.jsx
|           |— DesignerProfile.jsx
|           |— Home.jsx
|           |— Login.jsx
|           |— Path.jsx
|           |— Preview.jsx
|           |— Profile.jsx / Profile.css
|           |— Register.jsx
|       |— services/
|           |— api.js        # Axios instance with auth interceptor
|       |— App.jsx
|       |— main.jsx
|       |— index.css
|   |— .gitignore
|   |— package.json
|   |— vite.config.js
|   |— vercel.json
|— DATABASE/
|   |— backup.sql           # Full database dump
|   |— schema.sql          # Schema-only export
|— docs/
|   |— RUN-WAY.docx
|   |— RUNWAYpdf.pdf
|   |— API TESTING SSDoc1.docx
|— README.md
|— package.json
|— structure.txt
```

12. Database Schema Summary

users

user_id (PK, AUTO_INCREMENT)
username, email, password (bcrypt hash)
first_name, last_name, role (enum: visitor|designer|admin)
gender, date_of_birth, bio, brand_name, specialty, location
website, instagram, facebook, twitter, avatar_url
status (active|suspended), suspension_reason, suspension_end_date
moderated_by, moderated_at, created_at

designs

design_id (PK, AUTO_INCREMENT)
designer_id (FK → users.user_id)
title, description, image_url, season
status (active|hidden), moderation_reason, moderated_by, moderated_at
created_at, updated_at

likes

like_id (PK, AUTO_INCREMENT)
user_id (FK → users.user_id)
design_id (FK → designs.design_id)
UNIQUE(user_id, design_id)
created_at

ratings

rating_id (PK, AUTO_INCREMENT)
user_id (FK → users.user_id)
design_id (FK → designs.design_id)
rating (INT 1-5)
created_at

follows

follow_id (PK, AUTO_INCREMENT)
follower_id (FK → users.user_id)
designer_id (FK → users.user_id)
UNIQUE(follower_id, designer_id)
created_at

activity_logs

log_id (PK, AUTO_INCREMENT)
user_id (FK → users.user_id)
action_type (login|upload_design|like_design|rate_design|follow|update_profile|delete_design|suspend|...)
target_id, details, created_at

moderation_logs

log_id (PK, AUTO_INCREMENT)
admin_id (FK → users.user_id)
target_type (user|design), target_id
action (hide|unhide|suspend|unsuspend|delete), reason
created_at

user_notifications

notification_id (PK, AUTO_INCREMENT)
user_id (FK → users.user_id)
type, message, is_read (BOOLEAN)
created_at

13. Conclusion

RUN-WAY demonstrates a production-ready full-stack architecture combining a React SPA, Node.js REST API, distributed MySQL database (TiDB Cloud), and cloud media storage (Cloudinary). The platform emphasizes security through JWT authentication, bcrypt password hashing, parameterized SQL queries, and role-based access control. Real-time functionality via Socket.IO ensures a seamless social experience for fashion designers and enthusiasts alike.

Report generated for RUN-WAY Fashion Platform. Stack: React + Vite | Node.js + Express | TiDB Cloud | Cloudinary | Render | Vercel